

03. 시스템 아키텍처 명세서

인증, 서버, DB, 실시간 통신, 파일 저장 구조

프로젝트명	Realtime Doodle Relay
문서 기준일	2026-06-05
작성자	김윤섭
문서 상태	MVP 기준 개발 산출물

사진 업로드, 랜덤 라운드, 실시간 공동 드로잉, 채팅, 결과 갤러리, 다운로드 기능을 갖춘 미니 프로젝트

1. 전체 구조

```

[React Client]
- Firebase Client SDK
- Socket.IO Client
- Canvas API
- hand-drawn UI
  |
  | Firebase Login / ID Token
  v
[Firebase Authentication]
  |
  | token attached to HTTP/Socket
  v
[Render Web Service]
- Node.js
- Express REST API
- Socket.IO Server
- Firebase Admin SDK
  |
  | Mongoose / MongoDB Driver
  v
[MongoDB Atlas]
- users / rooms / images / rounds / results / chatMessages
- GridFS for original/result images
  
```

2. 기술별 책임

기술	책임	저장 여부
React	화면 렌더링, Canvas 조작, 채팅/드로잉 UI	브라우저 상태만 유지
Firebase Authentication	회원가입, 로그인, ID Token 발급	인증 사용자 정보 보관
Firebase Admin SDK	서버에서 ID Token 검증	저장 없음
Express	REST API, 업로드, 결과 조회/다운로드	저장 없음
Socket.IO	방 입장, 채팅, 드로잉, 라운드 이벤트 실시간 동기화	메모리 상태 일부 유지
MongoDB Atlas	서비스 데이터 저장	영구 저장
GridFS	원본 이미지와 결과 이미지 바이너리 저장	영구 저장
Render	백엔드 서버 실행 및 외부 URL 제공	로컬 파일 영구 저장 금지

3. 인증 흐름

1. 사용자가 프론트엔드에서 Firebase Authentication 으로 로그인합니다.
2. Firebase Client SDK 가 ID Token 을 발급합니다.
3. 클라이언트는 API 요청의 Authorization 헤더에 Bearer Token 을 포함합니다.
4. Socket.IO 연결 시 handshake.auth.token 에 같은 토큰을 포함합니다.
5. 서버는 Firebase Admin SDK verifyIdToken 으로 토큰을 검증합니다.

6. 검증된 uid 를 MongoDB users.firebaseUid 와 매핑합니다.

4. 실시간 통신 구조

Socket Namespace: default namespace
Room Unit: roomCode

client emit join-room -> server socket.join(roomCode)
client emit send-message -> server io.to(roomCode).emit(receive-message)
client emit draw-stroke -> server socket.to(roomCode).emit(draw-stroke)
server emit round-started -> room clients render selected image and timer
server emit round-ended -> room clients stop drawing and save result

5. 서버 메모리와 DB 분리

데이터	메모리	MongoDB
현재 접속 socketId	필요	선택
현재 라운드 타이머	필요	rounds 에 결과 기록
드로잉 좌표	실시간 브로드캐스트	MVP 에서는 저장 생략 가능
채팅 메시지	실시간 브로드캐스트	최근 50 개 저장 권장
원본 이미지	불필요	GridFS 저장
결과 이미지	불필요	GridFS 저장

6. Render 배포 제약

- Render 로컬 파일 시스템은 영구 저장소로 사용하지 않습니다.
- PORT 는 process.env.PORT 를 사용합니다.
- 무료 플랜은 비활성 상태에서 spin down 될 수 있으므로 실시간 시연 전 서버 상태를 확인해야 합니다.
- MVP 는 단일 Render Web Service 인스턴스 기준으로 설계합니다.
- 다중 인스턴스 확장 시 Socket.IO adapter 가 필요합니다.